

Day 8 Evidence Workbooklet

Expected-vs-Actual Debugging and Test Design

Engineering practice → observed route → expected route → likely cause → regression test

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	_____/34		Mastery check	secure / developing / needs support	

The sensor-kit checkout route is now being tested. Some routes look safe for one kit but fail for another. Today the focus is debugging: separate the observed behavior from the expected behavior, identify a likely cause, choose a minimal fix, and keep a test that would catch the bug again.

Engineering task	Use a repair log to decide whether a checkout route is trustworthy before the lab uses it.
Computational move	Compare expected and actual behavior, connect the mismatch to a code cause, and design tests that cover important routes.
Python focus	expected route, actual route, condition order, Boolean release condition, minimal fix, boundary and regression tests.
Evidence to keep	symptom, expected route, actual route, likely cause, minimal repair, test case, and support category.

Day 8 working rules

1. A symptom says what went wrong.
2. Expected behavior says what should have happened.
3. Actual behavior says what the code did.
4. A likely cause points to a condition or order problem.
5. A regression test keeps a known bug from quietly returning.

1 Separate symptom, cause, and repair

recognize debug

The checkout desk finds that some kits are routed incorrectly. A good debug note separates what happened, what should have happened, and what line likely caused it.

```
1 charge_percent = 82
2 calibration_label = "expired"
3 borrower_initials = "KM"
4
5 if charge_percent >= 80:
6     route = "release kit"
7 elif calibration_label != "current":
8     route = "calibration bench"
9 elif borrower_initials == "":
10    route = "borrower log needed"
11 else:
12    route = "hold for review"
```

Pts.	Prompt	My evidence
1	What route will the current code assign?	
1	What route should the kit receive if calibration is expired?	
1	Which line lets the kit release too early?	
1	Is the first problem a syntax error or a logic error?	

2 Build an expected-vs-actual table

trace debug

Use the same code. Complete the table and name the likely cause.

Pts.	Case	Expected route	Actual route	Likely cause
2	charge 82, expired calibration	calibration bench		
2	charge 76, current calibration	charge station		
2	charge 90, current calibration, blank initials	borrower log needed		
2	Explain which case best exposes the bug.			

3 Design tests before trusting a repair

[test](#) [explain](#)

A repair is not trustworthy just because it works for one case. Pick cases that cover the routes.

Pts.	Prompt	My test evidence
2	Give a test case that should route to "charge station".	
2	Give a test case that should route to "calibration bench".	
2	Give a test case that should route to "borrower log needed".	
2	Give a test case that should route to "release kit".	

Regression-test idea

A regression test is a small case you keep because it catches a bug that already happened once. If the bug comes back, that test should fail again.

4 Choose a minimal fix

[implement](#) [debug](#)

One safe repair is to require all release evidence before assigning "release kit". Complete the condition.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    route = "release kit"
elif charge_percent < 80:
    route = "charge station"
elif calibration_label != "current":
    route = "calibration bench"
elif borrower_initials == "":
    route = "borrower log needed"
else:
    route = "hold for review"
```

Pts.	Prompt	My response
2	Why does the release condition need three evidence checks?	
2	Which route will the expired-calibration case now receive?	
2	What risk remains if damage evidence is added later but not checked first?	
2	Name one test you would rerun after this repair.	

5 Write a repair log

debug test explain

Log field	My repair evidence
Symptom	
Expected behavior	
Likely cause	
Minimal fix	
Regression test	

Pts.	Debugging note
4	Explain why expected-vs-actual evidence is stronger than saying "the code is wrong." Use one route from the sensor-kit checkout example.
2	Circle the support route you would use next: symptom / expected route / actual route / cause / test case / minimal fix. Name one next action: _____

Bridge to Exam 1 and Day 10

Day 9 is Exam 1. Use expected-vs-actual evidence, minimal repair, boundary tests, and support-route notes as Exam 1 preparation. Day 10 returns to branch maps and test evidence after the exam and bridges into repeated cases and loops.

Scope boundary: keep the interface fixed

Do not rewrite the `def` line, parameter names, or exact output labels. Debug only the order of the indented decision rules. This page adds no points; the workbook score remains unchanged.

The provided wrapper below has the right inputs and output labels, but the rules are in the wrong order.

```
def badge_sheet_plan(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if total_badges <= 16:
        return "two sheets"
    elif total_badges <= 8:
        return "one sheet"
    else:
        return "print batch"
```

Total	Expected	Actual from wrong order	Evidence / likely cause
8	one sheet		
9	two sheets		
17	print batch		

Minimal repair	
Regression tests to keep	

Visible-check recovery loop

Run the editable file cell, run the visible checks, compare expected with observed, revise one rule, and rerun both cells. Do not change the setup or check cells to make a failure disappear.